THE UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Kiểm thử phần mềm - 22KTPM3

API TESTING - HW7

Thành Viên

ID	Full name
22127424	Nguyễn Phước Minh Trí

Giáo viên hướng dẫn

Trần Duy Hoàng

Trương Phước Lộc

Hồ Tuấn Thanh

Contents

Chapter 1

Group Information

Group ID: 07

Member Name	Student ID	Assigned Features	Status
Cao Uyển Nhi	22127310	- POST http://localhost:8091/favorites	Done
		- POST http://localhost:8091/brands	Done
		- POST http://localhost:8091/payment/check	Done
Lưu Thanh Thuý	22127410	- POST http://localhost:8091/users/login	Done
		- POST http://localhost:8091/users/register (user manage - admin only)	Done
		- PUT http://localhost:8091/users/{id} (user manage - admin only)	Done
Nguyễn Phước Minh Trí	22127424	- POST http://localhost:8091/products	Done
		- PUT http://localhost:8091/products/{productId}	Done
		- GET http://localhost:8091/products/{productId}/related	Done
Võ Lê Việt Tú	22127435	- POST http://localhost:8091/users/register	Done
		- GET http://localhost:8091/products	Done
		- GET http://localhost:8091/products/{productId}	Done
Trần Thị Cát Tường	22127444	- POST http://localhost:8091/messages	Done

Member Name	Student ID	Assigned Features	Status
		- POST http://localhost:8091/categories	Done
		- PUT http://localhost:8091/categories/{categoryId}	Done

Chapter 2

API Testing Summary

2.1 Scope and APIs Under Test

- **Base URL:** http://localhost:8091
- **Default header:** Content-Type: application/json
- **Auth header (when add, update product):** Authorization: Bearer
- **Login endpoints:**
  - **Admin:** POST /users/login

```
{
"email": "admin@practicesoftwaretesting.com",
"password": "welcome01"
}
```

2.1.1 API 1 – Create Product

- **Endpoint:** POST /products
- **Purpose:** Store new product in catalog.
- **Requirements & Headers:** Must be logged in (admin).
- **Always send Content-Type:** application/json.
- **Request Example:**

```
{
  "name": "Laptop",
  "description": "Gaming device",
```

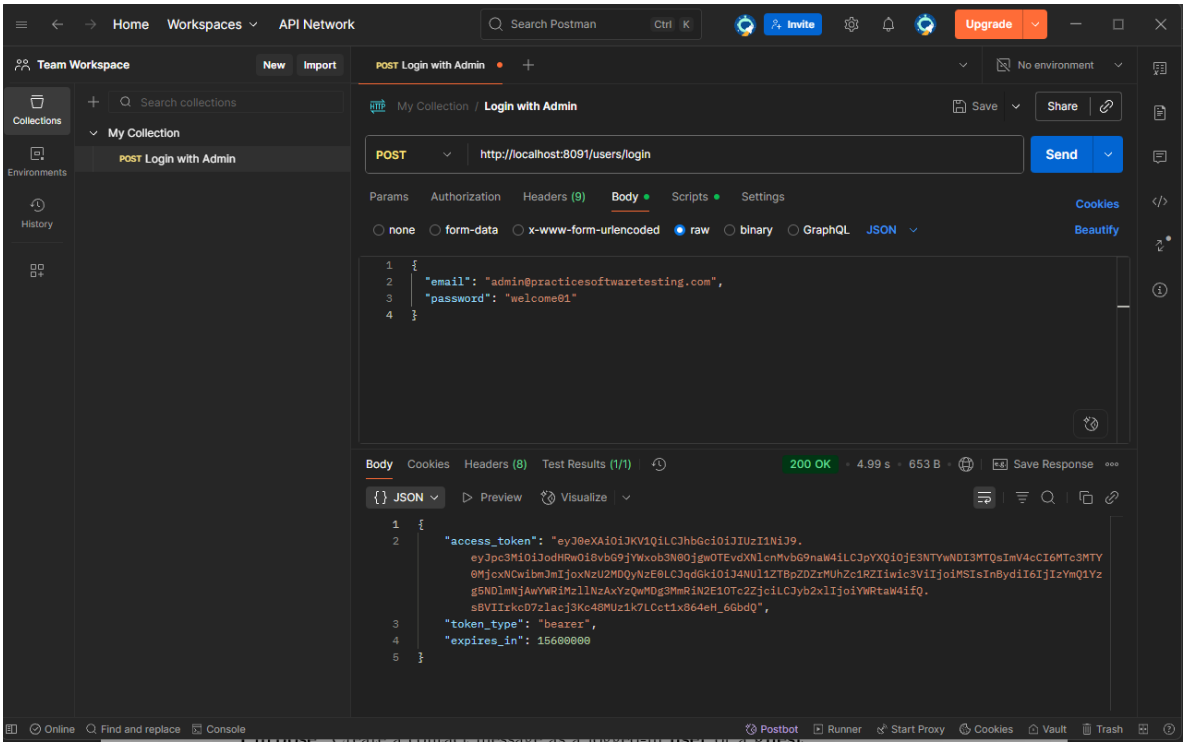

Figure 2.1: Login with admin


```
  "price": 1999.99,
  "category_id": 1,
  "brand_id": 1,
  "product_image_id": 1,
  "is_location_offer": 1,
  "is_rental": 0
}
```


- **Validations:**
  - **name:** required, < 256 chars.
  - **price:** > 0, max 2 decimals.
  - **category_id, brand_id, product_image_id must exist.**
  - **Flags** is_location_offer, is_rental in {0,1}.
- **EP & BVA Examples:**
  - **price:** {-1, 0, 0.01, 999999999}
  - **name:** {empty, 1 char, 255 chars, 256 chars}
  - **id:** {null, valid, non-existent, string}

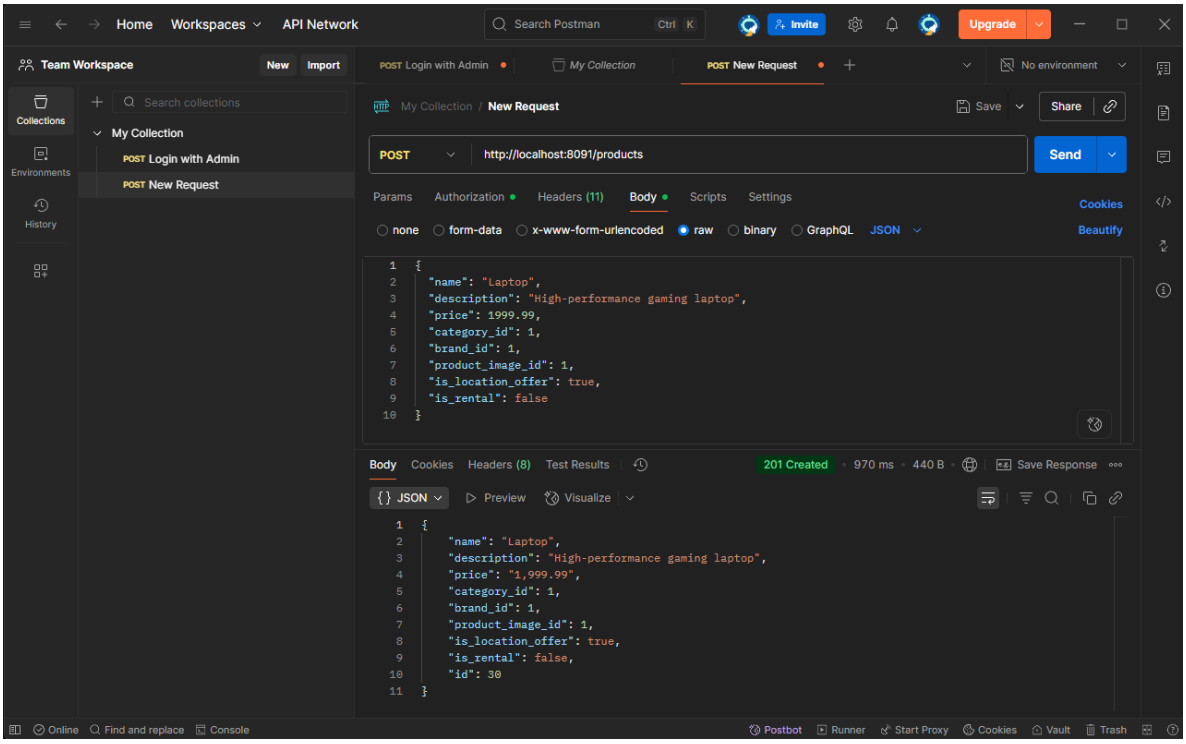

Figure 2.2: Add product

2.1.2 API 2 – Update Product

- **Endpoint:** PUT /products/{productId}
- **Purpose:** Update fields of existing product.
- **Requirements:**
  - Admin login required.
  - productId must exist.
- **Request Example:**

```
{
  "name": "Laptop Pro",
  "description": "Updated description",
  "price": 2299.99,
  "category_id": 1,
  "brand_id": 1,
  "product_image_id": 1,
  "is_location_offer": 0,
  "is_rental": 1
}
```

- **Validations:**

- productId path param must be integer > 0.
- Same rules as Create Product for fields.
- Empty/malformed body → 422/400.
- Updating to duplicate product not allowed.

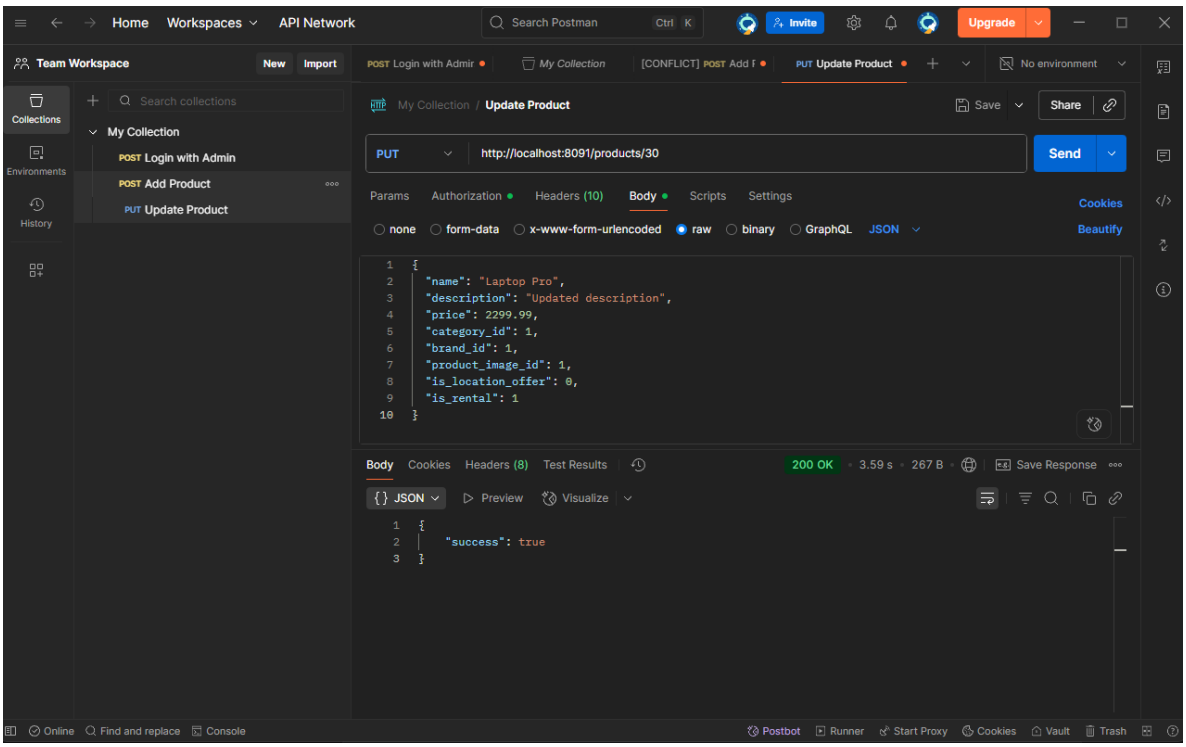

Figure 2.3: Update product

2.1.3 API 3 – Get Related Products

- **Endpoint:** GET /products/{productId}/related
- **Purpose:** Retrieve related products list.
- **Requirements:**
  - Auth optional, always Content-Type.
  - Response: array of products with nested brand, category, product_image.
- **Validations:**
  - productId must exist.
  - 404 if not found.
  - Schema of related products must match expected fields.

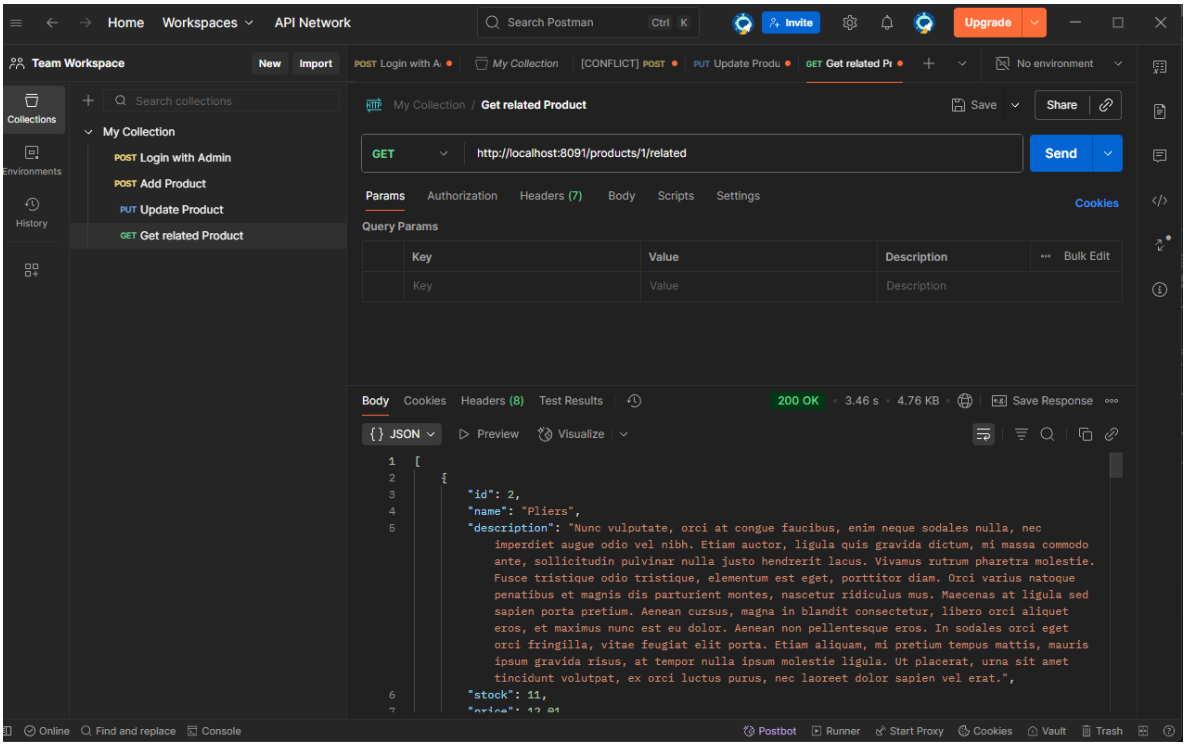

Figure 2.4: Get related product

2.2 AI-Assisted Test Case Generation

2.2.1 Approach

- Defined field rules (required/optional, length, type, uniqueness).
- Applied Equivalence Partitioning & Boundary Value Analysis.
- Prompted AI with: API spec, request/response examples, validation rules, and required test case format.
- Reviewed & refined AI-generated cases, added missing scenarios manually.
- Imported valid test cases into Postman collection.

2.2.2 Example Prompt

2.2.2.1 API 1 – Create Product

- You are a software testing assistant. I will provide API details, request/response examples, and validation rules. Generate at least 30 test cases in table format with columns: Test Case ID, Title, Preconditions, Inputs, Expected Result.
- API:

POST /products

- Purpose: Create new product. Fields: name (required, 255 chars), price (>0, 2 decimals), category_id (must exist), brand_id (must exist), product_image_id (must exist), is_location_offer in {0,1}, is_rental in {0,1}.

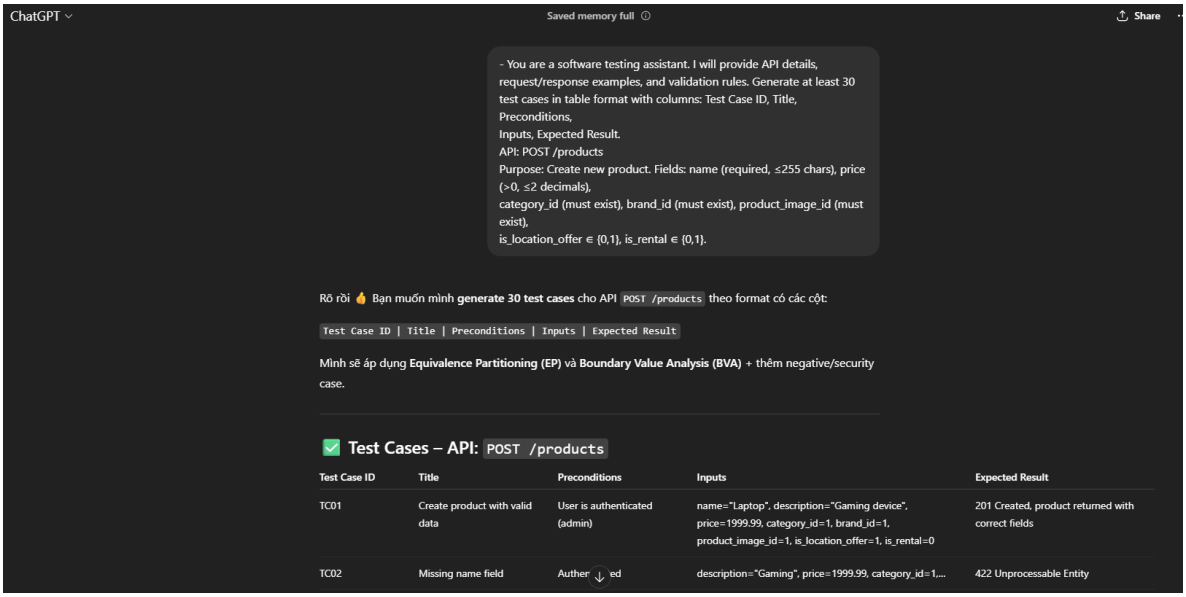

Figure 2.5: Example Prompt

2.2.2.2 API 2 – Update Product

- You are a software testing assistant. I will provide API details, request/response examples, and validation rules. Generate at least 30 test cases in table format with columns: Test Case ID, Title, Preconditions, Inputs, Expected Result.
- API:

POST /products/{productId}

- Purpose: Update product to the inventory.
- Fields: name (required, < 256 chars), price (>0, < 2 decimals), category_id (must exist), brand_id (must exist), product_image_id (must exist), is_location_offer in {0,1}, is_rental in {0,1}.

2.2.2.3 API 3 – Get Related Products

- You are a software testing assistant. I will provide API details, request/response examples, and validation rules. Generate at least 30 test cases in table format with columns: Test Case ID, Title, Preconditions, Inputs, Expected Result.
- API:

GET /products/{productId}/related

- Purpose: Retrieve related products for a given product ID.

2.3 Postman Workflow

- **Create collection** ‘My collection.
- **Add requests:** POST users/login , POST /products, PUT /products/{id}, GET /products/{id}/related.
- **Login to get tokens:**
  - Run POST /users/login with admin creds, copy access_token.
- **Set headers:**
  - Content-Type: application/json
- **Authorization:** Bearer (for admin APIs)
- **Set body:** raw JSON from test cases.
- **Run collection** with Postman.

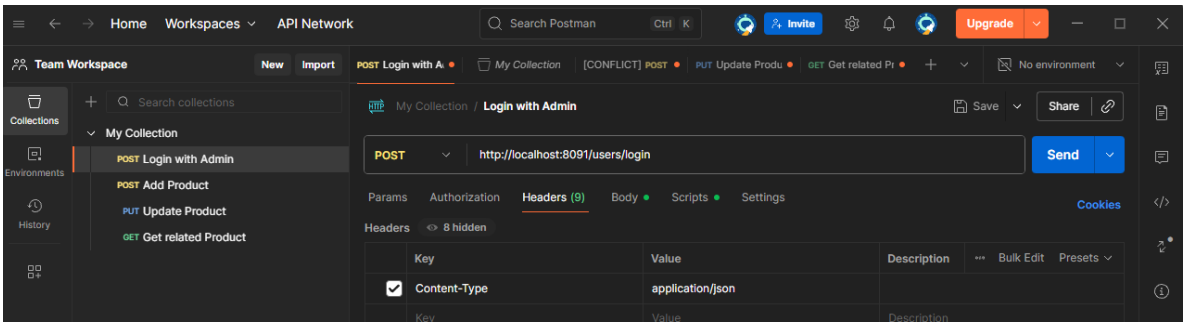

Figure 2.6: Postman Workflow

2.4 Test Execution Summary by Feature

Feature ID	Total Cases	Passed	Failed	Pass Rate
F01 – Create Product	30	24	6	80%
F02 – Update Product	30	28	2	93.3%
F03 – Get Related Products	30	27	3	90%
Overall	**90**	**63**	**27**	**87.76%**

Chapter 3

API Testing Integration into CI Workflow

3.1 Objective

- Run API tests automatically on every push/PR via GitHub Actions.
- Provide fast feedback to devs, detect regressions early, store artifacts.

3.2 Implementation Steps

1. Fork & clone repo.

Link: Github

2. Export Postman collection to api-tests/collections/toolshop.postman_collection.json.

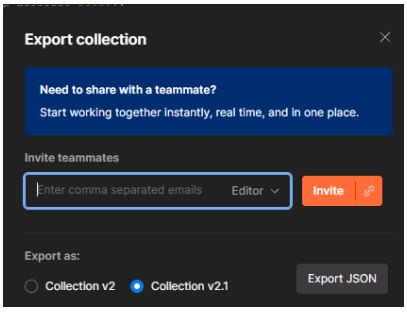

Figure 3.1: Export Postman collection

3. Create workflow .github/workflows/api-tests.yml:

- Checkout code.

– name: Checkout

```
      uses: actions/checkout@v4
```

- Start backend via docker-compose.

```
- name: Start containers
      run: docker compose up -d
```

- Seed DB.

```
- name: Run migrations and seed
      run: docker compose exec -T laravel-api php artisan migrate:fresh --
```

- Sanity check with GET /status.
- Install Newman + htmlextra reporter.

```
- name: Install Newman
      run: |
        npm install -g newman
        npm install -g newman-reporter-htmlextra
```

- Run collection:

```
newman run api-tests/collections/toolshop.postman_collection.json \
  --env-var baseUrl="http://localhost:8091" \
  --reporters cli,junit,htmlextra \
  --reporter-junit-export reports/junit.xml \
  --reporter-htmlextra-export reports/report.html
```

- Upload report.html + junit.xml as artifacts.

```
- name: Upload Newman Reports
      uses: actions/upload-artifact@v4
      with:
        name: newman-reports
        path: reports/
```

4. Commit & push to trigger workflow.

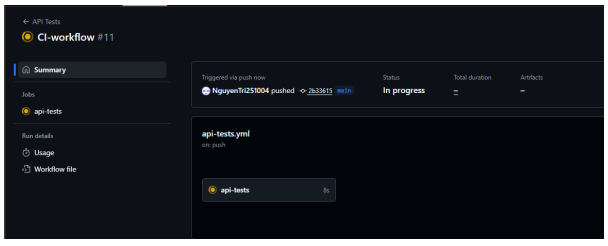

Figure 3.2: Commit & push to trigger workflow

3.3 Results

- Workflow succeeded.
- Artifacts:
  - junit.xml (machine-readable for CI dashboards).
  - report.html (interactive summary with pass/fail, response times).
- Evidence: downloaded artifact shows detailed test results.

3.4 Conclusion

- API tests integrated successfully into CI.
- Every push/PR now triggers:
- Start backend.
- Run Newman API suite.
- Store reports as artifacts.
- Provides continuous verification, fast feedback, and test traceability.

Chapter 4

Self-Evaluation

Criteria	Max Point	Self-assessment
1. API 1	30	30
1.1. Report	15	15
1.2 Test cases (30)	5	5
1.3. Screenshots on Testing Tools	5	5
1.4 Bugs	5	5
2. API 2	30	30
2.1. Report	15	15
2.2 Test cases (30)	5	5
2.3. Screenshots on Testing Tools	5	5
2.4 Bugs	5	5
3. API 3	30	30
3.1. Report	15	15
3.2 Test cases (30)	5	5
3.3. Screenshots on Testing Tools	5	5
3.4 Bugs	5	5
4. Well formatted	10	10
Total	100	100